

Safety Gate v2

Engineering Sprint Report

An 8-phase, tightly-scoped sprint that moved CalmGuide's safety evaluation ahead of expensive RAG/LLM work, added structured risk routing, fixed a real classifier bug, and built a hardened regression corpus — followed by an independent end-to-end audit that verified the results live.

Backend tests: 1578 → 1837 passing · Zero regressions at any phase

Prepared: September 2026

1 What This Sprint Was

CalmGuide's Moment Coach ran its two-layer safety check (a deterministic regex gate, then a heuristic fuzzy classifier) **after** building the RAG query, retrieving context, reading dossier/incident history, and constructing the full prompt — meaning every message, including genuine emergencies, paid for that entire pipeline before being safety-gated. This sprint's job was to fix that architecture properly: move safety first, make its output a structured, typed decision instead of an implicit boolean, and prove — empirically, not just by inspection — that the fix actually works and that normal-path performance wasn't harmed in the process.

✓ The core result, stated plainly

Emergency and high-risk messages now skip the RAG-query LLM call, RAG retrieval, and DB-context reads entirely — verified live via an independent concurrency probe and a real browser session that submitted a crisis message and received the correct escalation with zero LLM calls made. Normal-path requests still run RAG and DB-context concurrently (max(RAG,DB), not RAG+DB) — confirmed empirically: 0.35s measured against two 0.3s-delayed calls, not 0.6s.

Test suite, start to finish

Checkpoint	Backend tests passing	Note
Sprint start	1578	Baseline before any Safety Gate v2 work
After Phase 2 (decision model)	1599	+21, purely additive
After Phase 3 (fine categories)	1661	+62
After Phase 4 (dilution-bug fix)	1666	+5, plus one self-caught new false positive fixed
After Phase 5 (routing wired in)	1694	+14 — the phase that changed live behavior
After Phase 7 (observability)	1717	+16, including sign-off regression tests
After Phase 8 (regression corpus)	1837	+120 — dataset + integrity tests

Zero regressions at any checkpoint — every phase's own new tests were added on top of a fully green suite, re-verified fresh at every step.

2 Phase by Phase

PHASE 1

Architecture Inspection

Read the code, don't touch it — find the real problem before proposing a fix.

- Traced coach.py's actual request order and found safety ran last, not first — the core finding the whole sprint responds to.
- Found Check-In was already safety-first with no RAG step — a much smaller gap than Moment Coach's.
- Documented two duplicate category enums, no risk-level model, and the classifier's exact dilution bug shape — all fixed in later phases.

Deliverable: docs/SAFETY_GATE_V2_PLAN.md — no code changed.

PHASE 2

Structured SafetyDecision

Give the system one typed, explicit answer instead of a boolean.

- Consolidated two duplicate category enums (SafetyGateType / SafetyRiskCategory) into one canonical type via alias, avoiding a 50-site rename.
- Introduced RiskLevel (LOW/MODERATE/HIGH/EMERGENCY) and SafetyAction, plus a SafetyDecision dataclass with no unsafe implicit defaults.
- Purely additive — not wired into coach.py yet, so today's behavior provably didn't change.

PHASE 3

Fine-Grained Categories

*Know *why* a message is risky, not just that it is.*

- Added BREATHING_DIFFICULTY, FALL_OR_HEAD_INJURY, REDUCED_CONSCIOUSNESS, ACUTE_CHANGE, MEDICATION_RISK, ROUTINE_CAREGIVER_ISSUE.
- Kept the deterministic gate's own categories fully authoritative and untouched — refinement only sharpens the label after a trigger.
- MEDICATION_RISK required an explicit modal + dose-verb pattern so ordinary medication mentions never match.

PHASE 4

Fixed the Dilution Bug

The classifier missed typo'd emergencies embedded in ordinary sentences.

- Root cause: every fuzzy-match window was padded to a fixed size, diluting short typo'd phrases ("hart attack") inside longer messages below threshold.
- Fix: try every window size from the phrase length up to the padded size, not just the padded size alone — thresholds themselves never touched.
- The fix itself reintroduced a new false positive ("passed out flyers") — caught immediately via the red-team harness, fixed with a literal regex exclusion, not a broader threshold change.

A textbook case of "fix reveals a second bug" — caught and closed within the same phase.

PHASE 5

Safety Moved Before Everything Expensive

The phase that actually changes live behavior.

- Safety evaluation now runs immediately after profile/locale resolution — before the RAG-query LLM call, RAG retrieval, DB-context reads, and prompt construction.
- Phase 3's fine categories became independent triggers, not just post-hoc labels — phrases like “can't catch his breath” now actually escalate end-to-end for the first time.
- MEDICATION_RISK deliberately does NOT skip generation — the existing system prompt already refuses dosing decisions, and no approved static replacement exists yet (marked TODO(LOF-APPROVAL), a temporary rule pending clinical review).
- Normal-path asyncio.gather() concurrency preserved exactly — verified, not assumed.

PHASE 7

Safety Observability

Make the new architecture measurable, reusing existing patterns.

- New in-process counters (risk level, category, action, RAG/LLM skip rates, provider failover, response-guard repair/fallback usage), surfaced on the existing /health endpoint.
- Categorical labels and counts only — never raw caregiver messages or PHI, matching the project's existing safety-log convention.

PHASE 8

Fixed Engineering Regression Corpus

117 cases, generated from real code output, hardened against silent drift.

- 109 ENGINEERING_CONFIDENT cases (behavior follows directly from an existing rule) + 8 NEEDS_CLINICAL_REVIEW cases (genuinely ambiguous, deliberately excluded from pass/fail assertions).
- Explicitly labeled an engineering artifact, not a clinical validation dataset, throughout its own documentation.
- Hardened after initial delivery: the generator is now marked developer-utility-only, with an explicit warning against ever auto-regenerating expected values after a code change — exactly the failure mode (“bug → regenerate → tests still pass”) a regression corpus exists to prevent.

■ Phase 6 — deliberately not run

Clinician-reviewed RAG governance remains BLOCKED pending LOF approval throughout this entire sprint. Confirmed via a dedicated audit pass: zero clinician-governance fields or schema exist anywhere in the codebase — no accidental scope creep.

3 What The Dataset Itself Found

Building Phase 8's regression corpus wasn't just data entry — running every candidate case through the real system surfaced two genuine, previously-unknown issues, each spun off as its own scoped, un-started follow-up ticket rather than being fixed inline.

Finding	What was discovered	Status
Ambiguous breathing over-trigger	"His breathing seems a little different when he sleeps, is that normal?" escalates to a full 911-style EMERGENCY response via the classifier's fuzzy match — likely too aggressive for a mild, uncertain observation.	Ticketed, not fixed. Must not weaken detection for explicit distress phrasing.
French self-harm coverage gap	"je veux mourir" ("I want to die") is not detected by any existing French pattern; "j'ai envie de mourir" is. The dataset deliberately uses the phrase that actually works, rather than faking coverage.	Ticketed, not fixed. Needs a review of sibling constructions before any pattern change.

Both were reproduced again during the follow-up audit (Section 4) with identical results — confirmed stable findings, not one-off artifacts.

4 The Follow-Up Audit

After the 8-phase sprint closed, a separate, adversarial-minded end-to-end audit was run against the same codebase — explicitly scoped to READ / TEST / TRACE / VERIFY / DOCUMENT only, with no fixes permitted. Its job was to check whether the sprint's own claims actually held up under fresh, independent pressure, not to take them on faith.

What the audit re-verified, fresh

- Backend suite: 1837/1837 passing, run again from a clean state — matched the claimed baseline exactly.
- An independently-written concurrency probe (not part of the committed suite) measured 0.35s for a simulated LOW-risk request against two 0.3s-delayed dependencies — direct proof of max(RAG,DB), not RAG+DB.
- A real backend was stood up (SQLite-backed) and driven through the actual browser: submitting "I want to kill myself" through the live Moment Coach UI returned the correct 988 escalation text with zero LLM calls — the single strongest piece of evidence in the whole audit.
- The historical facility behavioral-card IDOR fix was re-tested in isolation — still holds, zero cross-facility access observed.

What the audit found that was new

- **HIGH — the entire sprint is uncommitted.** Every phase, all 1837 passing tests, exists only in the working tree, not in any git commit.
- **HIGH — /health's overall status doesn't factor in RAG availability.** Phase 7 added the data (a counter for RAG failures) but nothing yet reads it into the actual health computation or alerting — a live, partially-open descendant of the exact incident class (a silent RAG outage) that motivated building that observability in the first place.

- A handful of MEDIUM/LOW findings unrelated to safety logic: a fully broken frontend lint command (Next.js 16 removed it, no replacement configured), one piece of genuinely dead mobile code with a resulting translation gap, and minor formatting/lint drift.

■ One thing the audit disclosed about itself, on purpose

Mid-audit, a diagnostic probe appeared to show persistence totally failing for emergency messages — which would have been a Critical finding. It was chased to a definitive root cause: a flaw in the probe script itself (an in-memory SQLite database's default cross-connection isolation), not the product. This is documented in full in the audit report rather than quietly fixed and omitted — an audit's credibility rests on showing that kind of work, not just the clean number at the end.

✓ No Critical findings confirmed anywhere

Encryption, JWT fail-closed behavior, bcrypt salting, response-guard behavior, SSE streaming, provider failover semantics, and the product's own public claims were all checked and found consistent with what the code actually does — no overclaiming found anywhere in the codebase or its documentation.

5 Where Things Stand Now

Item	Status
Safety evaluation ordering (the core Phase 1 problem)	Fixed and verified live
Structured risk model + fine categories	Shipped, tested, in production code path
Classifier dilution bug	Fixed; the fix's own side effect was also caught and fixed
Safety observability (/health)	Shipped — but not yet wired into overall health/alerting
Regression corpus	117 cases, hardened against silent auto-regeneration
Ambiguous breathing over-trigger	Known, ticketed, not yet started
French self-harm coverage gap	Known, ticketed, not yet started
MEDICATION_RISK routing	Temporary rule, explicitly marked pending LOF/clinical review
Phase 6 (clinician RAG governance)	Still blocked pending LOF approval — confirmed untouched
Committing the sprint to git	Not yet done — the single most important next action

The engineering is sound. The most valuable next step isn't a new feature or a bug fix — it's making sure this work is actually saved.